

Flow Control Statements

Quite often in programs, we only want to do something provided that something else is true. As said in the previous lesson, programming involves examining a set of conditions and deciding what action to take based on those conditions.

Python's **if** statements allows you to examine the current state of a program and respond appropriately to that state. In this lesson, you'll learn how to write conditional tests (these allow you to check any condition of interest), learn how to write simple if statements and even learn how to create a more complex series of if statements to identify when the exact conditions you want are present.

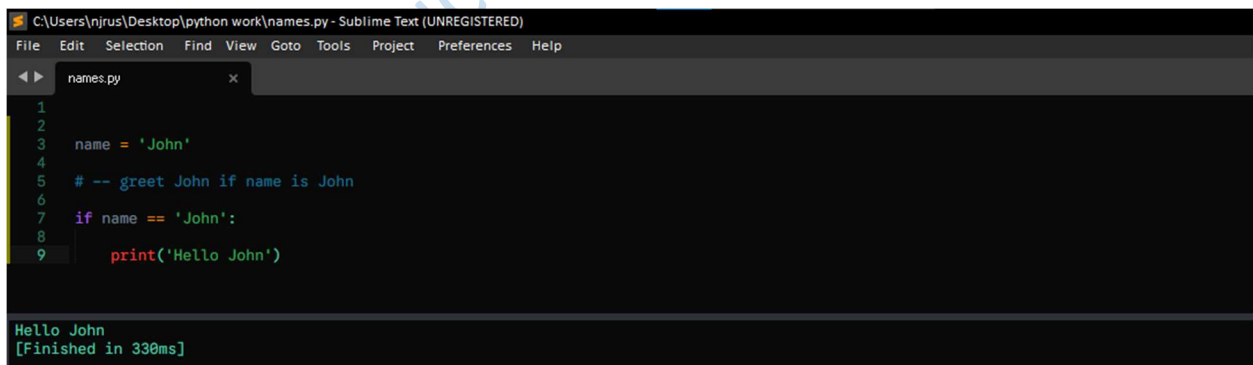
If Statements

The most common type of flow control statement is the **if statement**. The instructions contained in an if statement's clause are executed if the statement's condition is true. The clause is skipped if the condition is false. In plain English, you could read an **if statement** as, "**if this condition is true, execute the code in the clause.**"

In python, an if statement consist of the following:

- The **if** keyword
- A **condition**, that is, an expression evaluating to True or False.
- A **colon**
- An **indented block** (starting on the next line) called the if clause.

Consider the following program:



```
C:\Users\njrus\Desktop\python work\names.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
names.py
1
2
3 name = 'John'
4
5 # -- greet John if name is John
6
7 if name == 'John':
8
9     print('Hello John')

Hello John
[Finished in 330ms]
```

This program is doing a simple job. It prints **'Hello John'** if the value associated with **name** is John. When you run this program, python first assigns **John** to the variable **name**. This is done using the **assignment operator '='**. In line 7, python checks if the value stored in name **is equal to** John.



FROM BASICS TO JUNIOR PYTHON DEV: Conditionals (B)

If `name == 'John':`
True

Since the condition `name == 'John'` evaluates to true, python runs the instructions in the clause. i.e., python runs `print('Hello John')`.

Let's have a look at another similar program:

```
C:\Users\njrus\Desktop\python work\names.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
names.py x
1
2
3 name = 'James'
4
5 # -- greet John if name is John
6
7 if name == 'John':
8     print('Hello John')
9
[Finished in 247ms]
```

This program is similar to the previous one only that we have changed the value associated with **name**. But note the program did not run the instructions in the if statement's clause. To understand why, we need to revisit the condition under which the instructions should be run. Let's have a look at the condition:

`name == 'John':`

This condition checks if the value associated with name is **equal to** (==) John. However, the value associated with name is James, so this condition evaluates to False. This causes the instructions in the clause to be skipped.

else Statements

The else statement is used to define a block of code that is executed when the preceding if condition evaluates to False. It acts as a **fallback** or **default action** in conditional logic and does not have a condition.

```
C:\Users\njrus\Desktop\python work\names.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
names.py x
1 name = 'James'
2 # -- greet John if name is John
3 if name == 'John':
4     print('Hello John')
5 else:
6     print('Unknown User')
Unknown User
[Finished in 303ms]
```



FROM BASICS TO JUNIOR PYTHON DEV: Conditionals (B)

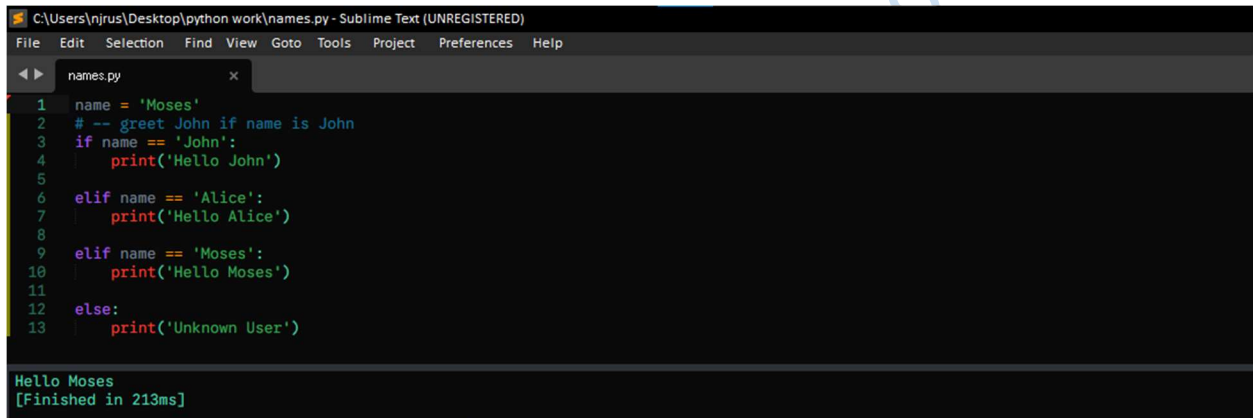
In this program, python checks the value associated with name and since the two aren't equal, the code in the else clause is executed and you see the output **Unknown User**.

elif Statements

The elif statement is an “else if” statement that always follows an **if** or another **elif** statement. It provides another condition that is checked only if any of the previous conditions were False. In code, an elif statement always consists of the following:

- The **elif** keyword.
- A **condition**, that is, an expression evaluating to True or False.
- An indented block of code starting on the next line.

Consider the following program:



```
C:\Users\njrus\Desktop\python work\names.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
names.py
1 name = 'Moses'
2 # -- greet John if name is John
3 if name == 'John':
4     print('Hello John')
5
6 elif name == 'Alice':
7     print('Hello Alice')
8
9 elif name == 'Moses':
10    print('Hello Moses')
11
12 else:
13    print('Unknown User')

Hello Moses
[Finished in 213ms]
```

The program is designed to greet a person if their personal name is assigned to a variable.

Python first checks the if condition. In this example, name **is not equal to** John, so the if clause is skipped. Python then moves to the elif condition and checks whether name is equal to Alice. Since this is False, python moves to the next elif condition. The third condition is True, so Python runs **print ("Hello Moses")**.

You can think of an elif statement as asking another question when the previous question was answered False:

```
if condition_1:
    # run this code if condition_1 is True
elif condition_2:
    # run this code if condition_2 is True
else:
    # run this code if all conditions are False
```



FROM BASICS TO JUNIOR PYTHON DEV: Conditionals (B)

An if statement can have one or more elif statements. The elif conditions are checked from top to bottom. Python stops at the first condition that evaluates to True and skips the remaining clauses.

Using multiple elifs

Sometimes there are more than two possible outcomes. For example, a program could respond differently depending on a person's age:

```
C:\Users\njrus\Desktop\python work\age.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help

age.py x
1 age = 20
2
3 if age < 13:
4     print("Child")
5 elif age < 18:
6     print("Teenager")
7 elif age < 60:
8     print("Adult")
9 else:
10    print("Senior")

Adult
[Finished in 244ms]
```

Here, Python checks the conditions in order. Because age is **20**, the first two conditions are False. The third condition, **age < 60**, is True, so Python prints "Adult" and stops checking the rest of the chain.

Order of conditions

Because Python stops at the first True condition, the order of an if/elif chain can change the result. Learn to put more specific conditions before broader conditions when necessary.

```
C:\Users\njrus\Desktop\python work\score.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help

score.py x
1 score = 85
2
3 if score >= 50:
4     print("Passed")
5 elif score >= 80:
6     print("Excellent")
7

Passed
[Finished in 252ms]
```

This prints "Passed" because **score >= 50** is already True. Python never reaches the elif condition. If you want to identify excellent scores separately, you'd want to check the more specific condition first:



FROM BASICS TO JUNIOR PYTHON DEV: Conditionals (B)

```
C:\Users\njrus\Desktop\python work\score.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help

score.py
1 score = 85
2
3 if score >= 80:
4     print("Excellent")
5 elif score >= 50:
6     print("Passed")
7 else:
8     print("Failed")
9

Excellent
[Finished in 213ms]
```

Using elif without else

An else statement is optional. You can use if and elif without an else when you only want to perform an action for certain conditions.

```
C:\Users\njrus\Desktop\python work\temperature.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help

temperature.py
1 temperature = 30
2
3 if temperature > 35:
4     print("It is very hot")
5 elif temperature > 25:
6     print("It is warm")
7

It is warm
[Finished in 249ms]
```

If neither condition is True, nothing is printed because there is no else clause.

Generally;

- **if** checks the first condition.
- **elif** checks another condition if all preceding conditions were False
- **else** runs when none of the preceding conditions were True.

Common mistakes

- Using `=` when you mean `==`. The single `=` assigns a value; `==` compares two values.
- Forgetting the colon (`:`) after if, elif, or else.
- Forgetting indentation. The code belonging to a condition must be indented.
- Using elif or else without an if. An elif must belong to an if/elif chain, and else must come after an if or elif.
- Putting conditions in the wrong order. Remember that Python stops at the first True condition.



Key takeaway

Conditional statements allow a program to make decisions. Use if to test the first condition, elif to test additional possibilities, and else as a final fallback when none of the conditions are True. The conditions are checked from top to bottom, and only the first True clause in the chain is executed.

From Basics to Junior Python Dev

